

Super Monitor for Vic20

Introduction

SMONV20 is an adaptation of the Supermon+64 program created by Jim Butterfield, the creator of the monitor found in the Commodore PET's ROM programs.

Mr. Butterfield passed away in 2007.

Several versions were ported to the Commodore Vic-20, but to my knowledge, always in a monochrome and minimalist form.

This adaptation attempt aims to modernize the monitor, make it more colorful, equip it with help screens, and add a few extra controls.

SMONV20 was created using the author's original source code, the 64TASS cross-assembler, a memory expansion cartridge (see the introductory screen), and an SD2IEC disk emulator. The VICE emulator was also used to facilitate and accelerate development.

This program is available in French and English versions, named respectively:

SVMONV20-FR.PRG and SVMONV20-EN.PRG.

***Note:** Each screen representation present in this document where created from a real Commodore Vic-20 using a screen capture software.*

Minimum system requirements:

The current version of SMONV20 loads into RAM at address 40960 (\$A000).

It is therefore necessary for the Vic20 computer to be equipped with a memory expansion card covering at least memory blocks 1 and 5.

Block 1 is necessary because the memory configuration (screen and color) changes when this memory block is active and SMONV20 has been programmed in this memory mode.

Block 5 is required to hold the SMONV20 program itself and must be RAM, since almost all of the program's variables are located within the \$A000 to \$BFFF range.

A floppy disk drive or a drive emulator—such as the SD2IEC, identified as device number 8 or 9—is required to save any programs or memory regions.

***N.B.:** Cassette player "#1" is not supported by SMONV20 and only works with 256-byte blocks.*

Launching SMONV20

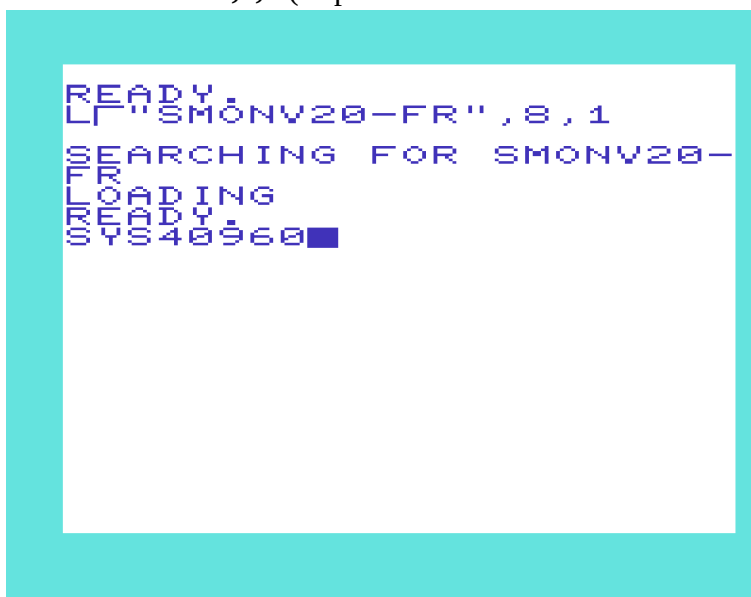
To load SMONV20 into memory, you must run the command:

LOAD "SMONV20.PRG",#1 (Replace the # with the drive number).

To launch the French or English version, you must run one of the following commands:

LOAD "SMONV20-FR.PRG",#1 (Replace the # with the drive number).

LOAD "SMONV20-EN.PRG",#1 (Replace the # with the drive number).



Upon startup, SMONV20 will display the following introductory screen, depending on the selected language.



List of commands

SMONV20 includes all the original commands from SUPERMON+64 and incorporates a few additional commands that have been added.

This section aims to provide examples of how all SMONV20 commands operate by presenting the resulting screens.

P.S. All commands work in both upercase and lowercase. They are shown here in upercase for the sake of clarity.

Note also that since all numeric inputs are assumed to be in hexadecimal, the use of the "\$" prefix to indicate this notation is optional in the SMONV20 environment, though it is used in the text for the sake of clarity.

Here, then, is the list of original commands from SUPERMON+64.

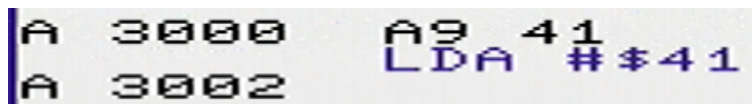
[A] – Assemble a program.

The "A" command places SMONV20 in assembler mode for as long as instructions are entered following the addresses.

Syntax: A Address Mnemonic [operand].

Example : A \$3000 LDA #\$41 [RETURN]

SMONV20 will respond by converting the instruction into hexadecimal and will wait for the next instruction at the next available address.



```
A 3000      A9 41
LDA #$41
A 3002
```

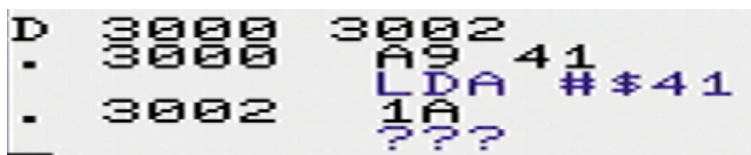
To exit assembler mode, press [RETURN] at the new address without entering an instruction.

[D] – Disassemble a program.

The "D" command displays memory contents converted into assembly language. The interpretation of the assembly code depends heavily on its validity. For example, if the memory region to be disassembled contains text, the resulting code will be unintelligible.

Syntaxe : D BEGIN [END]

Example : D \$3000 \$3002 [RETURN]



```
D 3000 3002
. 3000      A9 41
LDA #$41
. 3002      1A
          ???
```

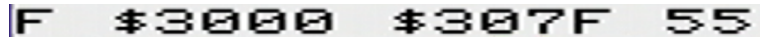
The "D" command, when used alone, disassembles the 16 bytes following the last operation or starting from the PC register (for the first use only). The command entry itself is cleared to allow for continuous display.

[F] – Fill (Populate memory).

The "F" command fills the specified memory region with a byte passed as a parameter. You must use the "M" command to view the result.

Syntaxe : F BEGIN END BYTE

Example : F \$3000 \$301F \$55 [RETURN]



```
F $3000 $307F 55
```

[M] – Memory (View/Edit the contents of a memory range).

The "M" command places SMONV20 in memory edit mode. Changes made to the hexadecimal values of each line are immediately applied to memory.

Syntaxe : M BEGIN [END]

Example: M \$3000 \$303F

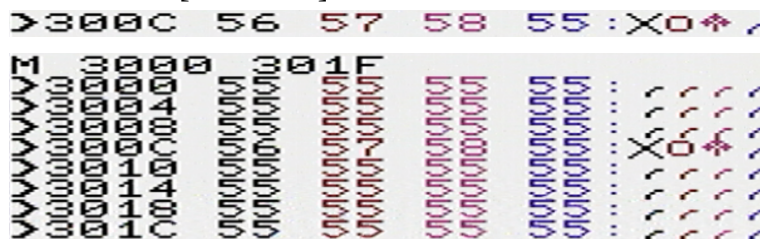


[>] - Modify memory content.

The ">" command allows you to modify four bytes of memory content, byte by byte.

Syntaxe : >address byte [byte] [[byte] [[byte]]]

Example: >\$300C 56 57 58 [RETURN]



[R] – Registers (View/Edit).

The "R" command displays the contents of the virtual registers.

Syntaxe : R

Example: R [RETURN]



```
R
SUPERMON FOR VIC20
; PC  SR  AC  XR  YR  SP
; A02D 30 08 00 62 F6
```

This screen showing the status of the register contents will appear automatically whenever the BRK instruction is encountered, or when the execution of the "J" command reaches a final RTS.

[;] - Modify registers.

The ";" command allows you to modify the contents of the virtual registers. The arguments for the ";" command must be entered in the order in which the registers appear via the "R" command. It is not necessary to include all the registers.

Syntaxe : ; Address PC [SR [AC [XR [YR [SP]]]]]

Example: ;3210 01 02 03 04 05

```
;3210 01 02 03 04 05
R
SUPERMON FOR VIC20
PC SR AC XR YR SP
; A02D 30 08 00 62 F6
```

For the following commands, please enter the small program below at address \$3000 using the "A" command.

```
LDA #$0D
JSR $FFD2
LDY #$0A
LDA #$41
JSR $FFD2
DEY
BNE $3000
BRK
```

Press [RETURN] on the blank line displayed at address \$3010. Here is what you should see on the screen.

```
A 3000 LDA #0D
A 3002 JSR D2 FF
A 3005 JSR $FFD2
A 3007 LDY #0A
A 3009 LDA #41
A 300B JSR D2 FF
A 300D JSR $FFD2
A 300F DEY
A 3010 BNE FA
BRK
```

[G] – Go (Launch a program into memory).

The Go command followed by an address initiates execution of instructions starting from that address. When the G command is issued without an argument, execution begins at the address contained in the virtual PC register.

Syntaxe : G [address]

Example: G \$3000 [RETURN]

```

G $3000
AAAAAAAAAA
SUPERMON FOR VIC20
PC SR AC XR YR SP
; 3011 32 41 03 00 05

```

Note the display of virtual registers created by the execution of the BRK instruction.

Note the content of the PC register, which holds the address following the end of your program.

Run the following command, which will change the instruction at address \$300F from BRK to RTS:

A \$300F RTS [RETURN]

Run the command "D \$3000 \$300F" to verify it.

```

D $3000 $300F
. 3000 LDA #0D
. 3002 LD D2 FF
. 3005 JSR $FFD2
. 3007 LDA #0A
. 3009 LDY #0A
. 300C LDA 41
. 300D LD D0 FF
. 300F JSR $FFD0
DEFY FA
BNE $3009
60
RTS <<---

```

[J] – Jump to Subroutine (Launch a subroutine in memory).

The "J" command executes a subroutine that ends with the RTS command.

When the J command is issued without arguments, execution begins at the address contained in the virtual PC register.

Syntaxe : J [address]

Example: J \$3000 [RETURN]

```

J
AAAAAAAAAA
SUPERMON FOR VIC20
PC SR AC XR YR SP
; 3000 32 41 00 00 F6

```

Note the display of the virtual registers resulting from the execution of the RTS instruction.

Note the content of the PC register, which holds the address where your program began.

[H] – Hunt (Search memory).

The H command searches for a byte or a sequence of bytes within a memory range and reports the addresses where they are found.

Syntaxe : H BEGIN END byte ... [byte]

Example: H \$B000 \$BFFF \$D2 \$FF [RETURN]

[T] – Transfer (Copy a memory range to another).

The "T" command allows a memory range to be copied (rather than transferred) to another.

Syntaxe : T BEGIN END DESTINATION

Example: T \$3000 \$3007 \$3010 [RETURN]

Run the following commands:

F \$3000 \$301f \$00

F \$3000 \$300f \$55

T \$3000 \$3007 \$3010

You will see the following screen:

[C] – Compare two memory ranges.

The "C" command compares two memory ranges byte by byte and reports the addresses where the data differs.

Syntaxe : C BEGIN END DESTINATION

Example: C \$3000 \$300f \$3010 [RETURN]

After typing the example provided for the T command, execute the following commands:

> \$3008 \$01 \$02 \$03 \$04

C \$3000 \$300f \$3010

You will see the following screen:

[\$] Convert a hexadecimal number to decimal, octal, and binary.

[+] Convert a decimal number to hexadecimal, octal, and binary.

[&] Convert an octal number to hexadecimal, decimal, and binary.

[%] Convert a binary number to hexadecimal, decimal, and octal.

The number conversion commands are quite simple to use. You simply need to precede the number to be converted with the prefix corresponding to the base in which you have chosen to enter it.

For example, if you enter the number 101 in each of the bases, you will obtain the following results:

```
$101
$0101
+257
&401
%100000001
+101
$0065
+101
&145
%1100101
&101
$0041
+65
&101
%1000001
%101
$0005
+5
&5
%101
```

These conversion prefixes can also be used when entering numbers in other commands within the SMONV20 environment.

Example: a +12288 jsr +65490

This example will result in the insertion of the "JSR \$FFD2" instruction at address "\$3000".

[S] – Save a memory range to disk.

The "S" command allows you to save a memory region to a file on a floppy disk.

N.B.: For this example, re-enter the example program for the ";" command, again at address \$3000.

Syntax : S "FileName",Drive#,BEGIN,END+1

Example: S "AFFICHE-A",8,3000,3010

The following message will appear.

```
S"AFFICHE-A",8,3000,3010
SAVING AFFICHE-A
```

Note that the end address must be incremented by 1 to include all bytes.

[L] – Load, load a file content into memory.

The "L" command loads the contents of a file on a floppy disk into a memory address.

Syntaxe : L "FileName",Drive#,Destination

Example: S "AFFICHE-A",8,3000

Type the command "F \$3000 \$300F \$00" to clear the program entered in the previous step, and type the command from this example to retrieve the program.

[@] - View floppy disk drive status.

The "@" command allows you to check the status of the currently selected drive or the drive specified by the parameter.

Syntaxe : @ [Drive#]

Exemples :

```
@
00, OK, 00, 00
L "GHJKGH", 8, 4000
SEARCHING FOR GHJKGH
@2, FILE NOT FOUND, 00, 0
```

[X] – Exit SMONV20 to Basic without clearing memory.

The "X" command allows you to exit BASIC without clearing the memory.

Syntaxe : X

Example:X

Enter the command "SYS40960" to return to "SMONV20".

List of commands added to SMONV20.

All added commands are prefixed with an exclamation mark "!".

Note : Where applicable, the screen content is preserved when using certain commands that alter it, such as "!D", "!G", and "!H".

![C] – Clear screen.

This command clears the screen. It was primarily added to assist users running an emulator where the [SHIFT]+[CLEAR] command cannot be executed from the keyboard.

![D] – Directory (Read a disk directory).

Lists the files on the currently selected disk and shows their status.

The [CTRL] key slows down the screen display.

![G] – Greetings (See the home screen).

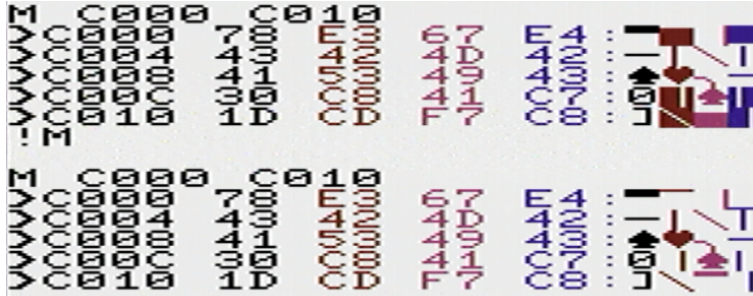
The "!G" command displays the welcome and credits screen (see page 2).

[!H] – Help (View a series of help screens covering all commands).

The "!H" command displays four help screens summarizing the use of all SMONV20 commands, followed by the credits screen.

[!M] – Mask bit 7 (Mask bit 7 when using the [M] command).

Inhibits or enables the masking of bit 7 for characters displayed on the right side of "M" or ">" command screens, and inhibits or enables the display of inverted characters.



[!8] – Select disk 8.

Select drive number 8 for the next "!D" command.

[!9] – Select disk 9.

Select drive number 9 for the next "!D" command.